

MonPoint MS

Migración de arquitectura monolítica a microservicios

Plan de Trabajo del Equipo

Documento de contextualización, metodología y cronograma

Proyecto	MonPoint MS — reingeniería arquitectónica de MonPointV2
Sistema base	MonPointV2 (Java 21 · JavaFX · Hibernate · MariaDB)
Equipo	Alejandro (Marquitos) · Victor · Hugo
Periodo	31 de agosto — 6 de noviembre de 2026 (10 semanas)
Versión	1.0 — documento de arranque

Contenido

1. Propósito de este documento
2. Contexto del proyecto
3. Arquitectura objetivo
4. Orden de implementación
5. Metodología de trabajo
6. Roles y responsabilidades
7. Cronograma
8. Asignación de microservicios
9. Puntos de control
10. Criterios de terminado
11. Herramientas y convenciones
12. Riesgos identificados
13. Qué necesita saber cada integrante

1. Propósito de este documento

Este documento define cómo va a trabajar el equipo durante las diez semanas del proyecto. No es una especificación técnica ni un manual: es el marco de trabajo. Establece qué se va a construir, en qué orden, quién responde por cada parte, con qué cadencia se sincroniza el equipo y bajo qué criterios se considera que algo está terminado.

Su función es eliminar ambigüedad. Cualquier integrante debe poder consultarlo y responder sin preguntar a nadie: *qué me toca, para cuándo, y cómo sé que ya acabé*. Los documentos de requisitos, arquitectura detallada y modelo de datos se entregan por separado.

Principio rector

Software funcional por encima de cualquier otra consideración. Ante conflicto entre cumplir el número de microservicios y entregar un sistema que arranca y opera correctamente, gana el sistema que funciona. Un servicio menos, bien construido y documentado, es mejor resultado que un servicio más a medio terminar.

2. Contexto del proyecto

2.1 De dónde partimos

MonPointV2 es un sistema de punto de venta para micro y pequeñas empresas, desarrollado como aplicación de escritorio monolítica. Actualmente opera de forma local y autónoma, sin dependencia de red.

Dimensión	Estado actual
Arquitectura	Monolito de capas: Controlador → DAO → Hibernate → MariaDB
Interfaz	JavaFX 21 con vistas FXML y temas CSS
Persistencia	Hibernate ORM 6.4 sobre MariaDB, base de datos única
Dominios	Usuarios, clientes, productos, proveedores, ventas, configuración
Despliegue	Ejecutable local (.jar / .exe), instalación manual

2.2 A dónde vamos

El proyecto consiste en rediseñar la arquitectura del sistema hacia un modelo de microservicios, rehacer el esquema de base de datos para que corresponda a esa arquitectura, e incorporar las tecnologías requeridas por la asignatura.

Requisito	Tecnología	Carácter
Autenticación por token	JWT (JSON Web Token)	Obligatorio

Requisito	Tecnología	Carácter
Descubrimiento de servicios	Netflix Eureka	Obligatorio
Contenerización	Docker + Docker Compose	Obligatorio
Punto único de entrada	Spring Cloud Gateway	Derivado de los anteriores
Orquestación de contenedores	Kubernetes	Opcional, no comprometido

Sobre Kubernetes

Queda fuera del alcance comprometido. Se evaluará únicamente si el sistema completo está estable antes de la semana 9. No se planifica trabajo, ni se asignan tareas, ni se estudia hasta que esa condición se cumpla.

2.3 Restricciones del equipo

La planeación está construida sobre estos supuestos. Si alguno deja de ser cierto, el plan debe revisarse.

- Disponibilidad estimada: 12 horas semanales por integrante.
- El microservicio de autenticación es obligatorio e invariante.
- El calendario incluye dos interrupciones previsibles: 16 de septiembre (miércoles) y 2 de noviembre (lunes).

3. Arquitectura objetivo

El sistema se descompone en seis microservicios de dominio más dos componentes de infraestructura. Los servicios no tienen peso equivalente: esto es una decisión de diseño, no una consecuencia del tiempo disponible. Un servicio con responsabilidad acotada y tres endpoints es un microservicio completo si cumple su función y respeta los criterios de terminado.

3.1 Microservicios de dominio

#	Servicio	Responsabilidad	Peso
1	ms-auth	Usuarios, roles, emisión y validación de JWT	Alto
2	ms-inventario	Productos, control de stock, categorías	Medio
3	ms-ventas	Ventas, detalle de venta, orquestación transaccional	Alto
4	ms-clientes	Clientes y proveedores (contrapartes comerciales)	Bajo
5	ms-reportes	Agregaciones, históricos, datos para gráficas	Bajo
6	ms-notificaciones	Eventos, alertas de stock bajo, comprobantes	Mínimo

3.2 Infraestructura

Componente	Función
eureka-server	Registro y descubrimiento de servicios
api-gateway	Punto único de entrada, enrutamiento, validación de JWT

3.3 Decisiones de diseño relevantes

Clientes y proveedores en un solo servicio

Ambas entidades son contrapartes comerciales con estructura casi idéntica: nombre, datos de contacto, dirección e identificador fiscal. Separarlas produciría dos servicios anémicos con duplicación de código. ms-clientes los maneja como dos agregados dentro de un mismo contexto.

Reportes como servicio de solo lectura

ms-reportes no posee datos propios: consulta a los demás servicios y agrega resultados. Esto elimina el costo de diseño de persistencia, migraciones y consistencia. Corresponde al patrón CQRS en su forma más simple.

Notificaciones delgado por diseño

ms-notificaciones tiene alcance deliberadamente mínimo: recibir eventos, registrarlos y disparar alertas de stock bajo. No incluye plantillas complejas, envío real de correo ni políticas de reintento. Este límite está definido desde el inicio y no debe ampliarse sin aprobación.

Comunicación síncrona entre servicios

La comunicación se resuelve con llamadas REST síncronas. No se implementa mensajería asíncrona, patrón Saga ni event sourcing. Cuando una operación cruce servicios —crear una venta descuenta stock en inventario— se maneja con compensación explícita y la limitación queda documentada. Introducir mensajería añadiría una tecnología nueva para todo el equipo sin ser requisito.

4. Orden de implementación

Los seis microservicios se diseñan, especifican y documentan desde la primera semana. Los seis aparecen en la arquitectura, en el modelo de datos y en el backlog. Lo que se estratifica no es el diseño: es el orden de implementación, derivado de dependencias técnicas reales.

Nivel	Servicios	Semanas	Criterio
Núcleo	ms-auth, ms-inventario, ms-ventas, eureka, gateway, compose	1 – 7	El sistema no es entregable sin esto. Todo lo demás depende de esta base.
Objetivo	ms-clientes, ms-reportes	6 – 8	Planificados y asignados. Se entregan salvo desviación mayor.

Nivel	Servicios	Semanas	Criterio
Extensión	ms-notificaciones	8 – 9	Especificado desde el inicio. Se implementa una vez el núcleo está estable.

La razón del orden es de dependencia, no de importancia. Sin autenticación, Eureka y Gateway operativos, ningún servicio de dominio puede desarrollarse, integrarse ni probarse. Por eso las primeras cuatro semanas se concentran en esa base: es ruta crítica pura.

Regla de alcance

Ningún servicio amplía su alcance por iniciativa individual. Si al construir un servicio se detecta una funcionalidad que valdría la pena añadir, se registra en el backlog y se prioriza en el planning siguiente. No se implementa sobre la marcha. El riesgo principal de este proyecto no es quedarse corto: es dispersarse.

5. Metodología de trabajo

El equipo trabaja con **Scrumban**: cadencia semanal fija tomada de Scrum, más límite estricto de trabajo en progreso tomado de Kanban.

La cadencia es semanal y no quincenal por una razón concreta: con dos integrantes aprendiendo tecnología nueva, un problema detectado a los catorce días cuesta el veinte por ciento del proyecto. A los siete días, la mitad.

5.1 Ceremonias

Ceremonia	Cuándo	Duración	Formato
Planning	Lunes	45 min	Síncrono
Check-in	Diario	5 min	Asíncrono, escrito
Review + Retrospectiva	Viernes o sábado	45 min	Síncrono, combinadas

El total de tiempo síncrono es de noventa minutos semanales sobre doce horas de trabajo: doce por ciento de sobrecarga. Es menos de lo que cuesta una semana mal ejecutada.

5.2 Formato del check-in diario

Tres líneas, escritas en el canal del equipo. Sin excepciones.

Ayer: [qué terminé, no qué toqué]
Hoy: [qué voy a terminar]
Bloqueo: [qué me detiene, o NINGUNO]

La distinción entre *terminé* y *trabajé en* es deliberada: obliga a reportar resultados, no actividad. «Sigo con el microservicio de productos» puede repetirse cuatro días seguidos sin que nadie detecte que no hay avance.

Regla de escalamiento

Ningún bloqueo puede durar más de 24 horas sin escalarse. Un integrante atascado tres días en silencio equivale a perder casi una semana-persona de la capacidad total del equipo. Pedir ayuda pronto no es debilidad: es el comportamiento esperado y explícitamente requerido.

5.3 Tablero y límite de trabajo en progreso

BACKLOG → LISTO → EN PROGRESO → REVISIÓN → HECHO
(máx. 1 / persona) (máx. 2)

Reglas del tablero

- Nadie inicia una tarjeta nueva teniendo una en progreso. Terminar antes de empezar.
- Si una tarjeta está bloqueada, se mueve a BLOQUEADO y se escala en el check-in. No se deja en progreso mientras se empieza otra cosa.
- Si REVISIÓN tiene dos tarjetas, revisar tiene prioridad sobre desarrollar. En equipos pequeños el cuello de botella suele ser la revisión, no el desarrollo: una tarjeta esperando cuatro días a ser revisada todavía no existe.
- Una tarjeta solo llega a HECHO si cumple íntegramente los criterios de terminado.

Por qué el límite de trabajo en progreso

Ataca el patrón más destructivo en equipos pequeños sin experiencia: tres cosas al sesenta por ciento y ninguna al cien cuando llega la fecha de entrega. Terminar una cosa vale más que avanzar en tres.

6. Roles y responsabilidades

Integrante	Rol	Responsabilidades
Alejandro	Product Owner Tech Lead Facilitador	Prioriza el backlog y define alcance. Decide arquitectura. Resuelve bloqueos técnicos. Interfaz con el profesor. Facilita las ceremonias. Owner de servicios.
Victor	Developer Owner de servicios	Desarrollo de los servicios asignados. Revisión de los pull requests que le corresponden. Documentación de sus servicios.
Hugo	Developer Owner de servicios	Desarrollo de los servicios asignados. Revisión de los pull requests que le corresponden. Documentación de sus servicios.

6.1 Sobre la figura del revisor

Cada servicio tiene un owner, que lo construye y responde por él, y un revisor, que no es un aprobador pasivo. El revisor tiene tres obligaciones concretas:

- Revisar los pull requests en menos de 24 horas.

- Ser capaz de ejecutar el servicio localmente.
- Ser capaz de cubrir el servicio si el owner se satura o se ausenta.

Esta última es la razón principal de la figura: ningún servicio debe depender de una sola persona.

7. Cronograma

Diez semanas, del lunes 31 de agosto al viernes 6 de noviembre de 2026. Un sprint por semana.

Sem	Fechas	Sprint	Entregable	Servicios activos
1	31 ago – 6 sep	S0	Entorno de desarrollo, onboarding, decisiones de arquitectura	—
2	7 – 13 sep	S1	ms-auth funcionando de forma aislada, JWT emitiéndose	auth
3	14 – 20 sep	S2	Eureka y Gateway operativos, auth registrado	auth + infra
4	21 – 27 sep	S3	Docker Compose levantando el conjunto · Control 1	auth + infra
5	28 sep – 4 oct	S4	ms-inventario completo (transición a trabajo paralelo)	inventario
6	5 – 11 oct	S5	ms-ventas y ms-clientes en paralelo	ventas, clientes
7	12 – 18 oct	S6	Cierre de ms-ventas y ms-reportes · Control 2	ventas, reportes
8	19 – 25 oct	S7	ms-notificaciones e integración entre servicios	notificaciones
9	26 oct – 1 nov	S8	Cliente JavaFX y pruebas end-to-end · Control 3	todos
10	2 – 6 nov	S9	Estabilización, documentación y entrega	—

Notas de calendario

- El 16 de septiembre cae miércoles, dentro de la semana de infraestructura, que es técnicamente la más densa. Esa semana se planifica con carga reducida.
- El 2 de noviembre cae lunes, dentro de la semana de estabilización. El impacto es absorbible.
- La semana 10 es buffer explícito, no trabajo planificado. Su función es absorber el desfase que siempre ocurre. Si se necesita para terminar funcionalidades, el proyecto ya está en problemas.
- Las fechas de exámenes parciales no están definidas. Se ajustará el plan conforme se conozcan.

7.1 Fases de trabajo

Semanas 1 a 4 — Programación en conjunto

Los tres integrantes trabajan sobre el mismo código, no en paralelo. Sesiones conjuntas donde uno escribe y los otros dos participan activamente, rotando cada veinte o treinta minutos.

Puede parecer ineficiente —tres personas produciendo el resultado de una— pero no lo es en este contexto. El cuello de botella de las primeras semanas no es capacidad de producción: es conocimiento. Trabajar en paralelo desde la semana dos produciría tres implementaciones divergentes que habría que rehacer. Al terminar la semana cuatro el equipo tiene infraestructura funcionando, una plantilla replicable y tres personas que ya construyeron un microservicio completo.

Semanas 5 a 8 — Trabajo paralelo con ownership

Cada integrante desarrolla los servicios de los que es owner y revisa los servicios que le corresponden como revisor.

Semanas 9 y 10 — Trabajo conjunto

Integración y estabilización. Los problemas de integración cruzan fronteras de servicio y no se resuelven con ownership individual.

8. Asignación de microservicios

Cada integrante es owner de dos servicios y revisor de dos. La asignación forma los tres binomios posibles del equipo, de modo que todos trabajan con todos y todos conocen varias partes del sistema.

Servicio	Owner	Revisor
ms-auth + api-gateway	Alejandro	Victor
ms-inventario	Victor	Hugo
ms-ventas	Alejandro	Hugo
ms-clientes	Hugo	Victor
ms-reportes	Victor	Alejandro
ms-notificaciones	Hugo	Alejandro

Criterio de la asignación

- ms-ventas es el servicio con orquestación distribuida: crear una venta implica descontar stock en ms-inventario y notificar a ms-notificaciones. Es donde aparecen los problemas de consistencia entre servicios, y por eso queda con quien tiene el contexto arquitectónico completo.
- ms-inventario es el primer servicio que se desarrolla en paralelo y sirve como primera implementación autónoma tras la fase conjunta.

- Los servicios de menor peso —clientes, reportes, notificaciones— se distribuyen para equilibrar la carga total sin concentrar riesgo.

9. Puntos de control

Tres controles con criterios objetivos. Su función es detectar desviación temprano, cuando todavía hay margen de corrección, en lugar de descubrirla en la última semana.

Control 1 — Fin de la semana 4 (crítico)

Criterio: un endpoint protegido, accesible a través del Gateway, autenticado con JWT, con el servicio registrado en Eureka, y todo el conjunto levantado con docker compose up desde cero.

Resultado	Acción
Cumplido	Continuar según el plan.
Parcial (falta Docker)	Continuar; Docker se completa durante la semana 5.
No cumplido	ms-notificaciones pasa a estado especificado, no implementado. La semana 5 se reasigna a completar infraestructura.

Control 2 — Fin de la semana 7

Criterio: ms-auth, ms-inventario y ms-ventas funcionando e integrados entre sí.

Resultado	Acción
Cumplido	Continuar con ms-notificaciones en la semana 8.
No cumplido	Congelar el alcance en cinco servicios y dedicar la semana 8 a estabilizar el núcleo.

Control 3 — Fin de la semana 9

Criterio: sistema completo funcionando end-to-end desde el cliente.

Si no se cumple, la semana 10 se dedica íntegramente a que funcione lo que ya existe. No se agrega nada nuevo en la última semana bajo ninguna circunstancia.

10. Criterios de terminado

Sin un criterio explícito, «terminé mi microservicio» significa cosas distintas para cada persona, y la diferencia se descubre en integración, cuando ya es cara. Estos criterios son negociables *ahora*; dejan de serlo cuando arranque la semana 2.

10.1 Definición de Terminado (Definition of Done)

Un microservicio está terminado cuando cumple todos los criterios:

Funcional

- ☐ Endpoints REST implementados según la especificación del backlog
- ☐ Validación de entrada en todos los endpoints
- ☐ Manejo de errores con códigos HTTP correctos (no todo 200 ni todo 500)
- ☐ Respuestas JSON consistentes con el contrato acordado

Integración

- ☐ Registrado correctamente en Eureka
- ☐ Accesible a través del API Gateway
- ☐ Valida JWT en los endpoints protegidos
- ☐ Endpoints públicos y protegidos explícitamente diferenciados

Persistencia

- ☐ Esquema de base de datos creado y versionado
- ☐ Migraciones ejecutables desde cero
- ☐ Sin credenciales escritas directamente en el código

Contenerización

- ☐ Dockerfile funcional
- ☐ El servicio levanta correctamente con docker compose up
- ☐ Variables de entorno externalizadas

Calidad

- ☐ Al menos una prueba por endpoint principal
- ☐ Código revisado y aprobado por el revisor asignado
- ☐ Sin código comentado ni impresiones de depuración

Documentación

- ☐ Endpoints documentados (OpenAPI o markdown)
- ☐ README del servicio con instrucciones de ejecución local

10.2 Definición de Listo (Definition of Ready)

Una historia no entra a un sprint si no cumple estos criterios. El propósito es evitar que alguien tome una tarjeta y descubra a mitad de semana que no sabe qué se espera de él.

- Tiene criterios de aceptación escritos.
- Tiene estimación asignada.
- Sus dependencias están identificadas y resueltas o planificadas.
- El owner asignado entiende qué debe construir.

11. Herramientas y convenciones

Función	Herramienta	Justificación
Backlog y tablero	GitHub Projects	Integrado con el repositorio. Vincula issues con commits y pull requests, lo que produce trazabilidad de requisito a código sin esfuerzo adicional.
Control de versiones	GitHub	Repositorio ya existente del proyecto.
Comunicación	Canal del equipo	El que el equipo ya usa. No se introduce herramienta nueva.
Documentación	Markdown en /docs	Versionada junto al código, de modo que no se desincroniza.
Diagramas	Mermaid o draw.io	Mermaid se renderiza de forma nativa en GitHub.

11.1 Convenciones de trabajo

- Una rama por historia, nombrada con el número de issue.
- Ningún cambio entra a la rama principal sin pull request aprobado por el revisor.
- Los commits referencian el issue correspondiente.
- Cada servicio vive en su propio directorio con su Dockerfile y su README.
- Las credenciales y configuraciones sensibles se manejan por variables de entorno, nunca en el código ni en el repositorio.

12. Riesgos identificados

Riesgo	Prob.	Impacto	Mitigación
Infraestructura no lista al terminar la semana 4	Media-alta	Crítico	Control 1; trabajo conjunto concentrado en infraestructura durante las primeras cuatro semanas.
ms-ventas excede lo estimado por consistencia distribuida	Alta	Alto	Asignado al integrante con contexto arquitectónico; comunicación síncrona simple; sin patrón Saga.
Servicios del nivel objetivo quedan a medias	Media	Alto	Criterios de terminado estrictos; regla de terminar antes de empezar.
El cliente JavaFX consume más tiempo del previsto	Media	Medio	Se adaptan solo los módulos críticos. No se rediseña la interfaz.
La curva de aprendizaje resulta más lenta de lo estimado	Media	Alto	Trabajo conjunto en la fase inicial; el Control 1 lo detecta en la semana 4.

Riesgo	Prob.	Impacto	Mitigación
Sobre-ingeniería en los servicios de alcance reducido	Media	Medio	Alcance por servicio definido y cerrado en el backlog.
Bloqueos no comunicados	Alta	Alto	Check-in diario obligatorio; regla de escalamiento en 24 horas.

Riesgo principal

El de mayor probabilidad por impacto es ms-ventas. Es el único servicio que coordina operaciones sobre datos que no le pertenecen. La decisión de resolverlo con llamadas síncronas y compensación explícita, en lugar de patrones distribuidos avanzados, es deliberada: la solución teóricamente correcta es demasiado cara para este calendario, y la solución simple es defendible si se documentan sus límites.

13. Qué necesita saber cada integrante

Resumen operativo. Si solo se lee una sección de este documento, que sea esta.

13.1 Lo que aplica a todos

- **Cadencia:** planning los lunes, review y retrospectiva al cierre de la semana, check-in escrito todos los días.
- **Límite de trabajo:** una sola tarjeta en progreso por persona. Terminar antes de empezar.
- **Bloqueos:** se comunican dentro de las primeras 24 horas. Sin excepción.
- **Revisión:** los pull requests asignados se revisan en menos de 24 horas y tienen prioridad sobre el desarrollo propio.
- **Terminado:** una tarjeta llega a HECHO solo si cumple los criterios de la sección 10.
- **Alcance:** no se amplía el alcance de un servicio por iniciativa individual. Se registra en el backlog y se prioriza en el planning.

13.2 Semanas 1 a 4

Se trabaja en conjunto sobre el mismo código. No hay asignación individual todavía. El objetivo de esta fase no es producir volumen: es que los tres integrantes terminen sabiendo construir un microservicio completo con Spring Boot, registrarlo en Eureka, exponerlo por el Gateway, protegerlo con JWT y levantarlo con Docker. Todo lo demás depende de que esto se logre.

13.3 A partir de la semana 5

Se activa la asignación de la sección 8. Cada integrante desarrolla los servicios de los que es owner y revisa los que le corresponden. El servicio construido en conjunto durante la fase inicial funciona como plantilla: la estructura de paquetes, la configuración de Eureka, el Dockerfile y el manejo de JWT se replican, no se reinventan.

13.4 Preparación previa recomendada

El material de estudio asignado debe revisarse durante la semana 1. Los temas mínimos por orden de utilidad:

- **Spring Boot:** estructura de un proyecto, inyección de dependencias, controladores REST.
- **Spring Data JPA:** repositorios y mapeo de entidades. Es el punto con mayor continuidad respecto al Hibernate ya usado en MonPointV2.
- **Conceptos de JWT:** qué es un token, cómo se firma y cómo se valida.
- **Docker:** imagen contra contenedor, Dockerfile, docker compose.
- **Netflix Eureka:** qué problema resuelve el descubrimiento de servicios.

Documento de trabajo, versión 1.0. La especificación de requisitos, el product backlog, la arquitectura detallada y el modelo de datos se entregan como documentos independientes. Este plan se revisa al cierre de cada punto de control y en cualquier momento en que alguno de los supuestos de la sección 2.3 deje de ser cierto.